

Módulo 3: Funciones y programación “pythonic”

Contenidos clave

Funciones, argumentos posicionales/nombrados, *args/kwargs:**

- Funciones: Una función es un bloque de código que encapsula una tarea específica, se puede reutilizar y puede devolver un valor; recibe información mediante parámetros (variable definida en la declaración de la función).
- Argumentos posicionales: Se asignan a los parámetros según el orden en que aparecen, son apropiados cuando tienen un significado evidente, el orden es claro y la función tiene pocos parámetros.
- Argumentos nombrados: (keyword arguments) se asignan explícitamente utilizando el nombre del parámetro, su principal ventaja es la legibilidad y permite que el orden de los argumentos no sea relevante, una misma llamada puede combinarlos con los posicionales, pero los últimos deben aparecer antes que los nombrados.
- Parámetros por defecto: un parámetro puede tener un valor predeterminado, lo que permite que el argumento sea opcional, se usa para configurar el comportamiento de una función sin obligar al usuario a proporcionar todos los valores.
- ***args**: permite que una función reciba una cantidad variable de **argumentos posicionales**, el operador “*” realiza un proceso llamado packing que agrupa los argumentos posicionales restantes en una tupla.
- ****kwargs**: permite recibir una cantidad variable de **argumentos nombrados**, el operador “**” agrupa esos argumentos en un diccionario.

Lambdas, closures y decoradores:

- Funciones como objetos de primera clase: (first-class object), esto significa que una función puede: almacenarse en una variable, pasarse como argumento, devolverse desde otra función, almacenarse en estructuras de datos y utilizarse como cualquier otro objeto.
- Lambda: es una función anónima que normalmente se utiliza para representar una operación pequeña: no necesita un nombre mediante def, está diseñada para expresiones simples, puede recibir múltiples parámetros, produce implícitamente el resultado de su expresión. / Forma compacta de crear una función para utilizarla en un contexto inmediato. Son frecuentes con funciones como sorted(), map(), filter(), etc.
- Closures: es una función que conserva acceso al entorno donde fue creada, ocurre cuando existe una función externa, dentro de ella se define una función, la función interna utiliza variables de la función externa y la función interna continúa existiendo después de que la función externa terminó. Los closures permiten crear funciones configuradas dinámicamente, la función interna recuerda los valores de su entorno léxico.
- Decoradores: es un mecanismo que permite modificar o extender el comportamiento de una función o clase sin modificar directamente su código fuente. Se basan en dos conceptos:
 1. Las funciones son objetos de primera clase.
 2. Una función puede recibir otra función y devolver otra función.

Se usan para implementar comportamientos como: autenticación, autorización, logging, medición de tiempo, validaciones, caché, control de permisos y manejo de transacciones.

Iteradores, generadores, comprensiones:

- Iterables: es un objeto que puede proporcionar sus elementos uno a uno. Es algo sobre lo que se puede realizar una iteración, cuando Python necesita recorrer un iterable, obtiene un *iterador*.
- Iteradores: es un objeto que representa un estado de recorrido y proporciona los elementos uno por uno.
- Generadores: es una forma especial de crear iteradores de manera sencilla, se construyen utilizando *yield*, este suspende temporalmente la ejecución en lugar de terminarla, cuando posteriormente se solicita otro valor, la ejecución continúa desde donde se había detenido. Es decir, es especialmente útil cuando se tiene que procesar grandes cantidades de datos, en vez de generar todo, almacenar todo y procesar, se puede generar y procesar, generar y procesar.
- Comprensiones / Comprehensions: son una característica sintáctica de Python que permite construir colecciones de forma compacta y declarativa. Su filosofía es: "Quiero construir una colección a partir de otra aplicando una transformación y /o condición". Principalmente existen:
 1. List comprehensions: produce una lista, puede incorporar una condición, expresa una transformación de manera clara y concisa.
 2. Dictionary comprehensions: permite construir diccionarios mediante una expresión. Es útil cuando una colección de entrada puede transformarse directamente en pares: clave-valor.
 3. Set comprehensions: permite construir conjuntos, dado que los set eliminan duplicados, el resultado contiene únicamente valores únicos.

- **Context managers (with):** es un mecanismo que permite administrar correctamente un recurso durante un bloque de código, garantiza que ciertas operaciones de entrada uso salida/limpieza se realicen correctamente, incluso cuando ocurre una excepción.

- with: es una instrucción que permite utilizar un context manager, con el, Python ejecuta automáticamente operaciones necesarias para: entrar al contexto, ejecutar el bloque, salir del contexto y realizar la limpieza correspondiente.
- `__enter__` y `__exit__`: internamente un context manager utiliza estas dos cuando encuentra with recurso: , entra, ejecuta el bloque y realiza la limpieza o manejo de excepciones.

- **Pythonic:** Significa escribir código siguiendo las convenciones, características y filosofía propias de Python, buscar código: legible, simple, expresivo, idiomático, mantenible y que aproveche correctamente las características del lenguaje.

Objetivos

- Diseñar APIs de funciones claras y expresivas.

Una Api de función es la forma en que otras partes del programa interactúan con una función: qué recibe, qué devuelve y cómo se utiliza. En el código utilizamos type hints, valores por defecto y nombres descriptivos para hacer esa interfaz clara. Para ejemplificar:

```
9 def reintentar(intentos_max: int = 3, retraso: float = 0.5) -> Callable:
```

En esta función, se indica que `intentos_max` recibe un entero, `retraso` un número decimal y ambos tienen valores por defecto y `-> Callable` devuelve algo que puede ser llamado, en este caso el decorador.

```
56 @reintentar(intentos_max=2, retraso=0.2)
```

La interfaz es bastante expresiva porque por el nombre de los parámetros se entiende bien qué hace cada uno.

Los nombres descriptivos, parámetros con valores por defecto y anotaciones de tipo nos ayudan a conocer qué recibe y qué devuelve cada función y facilita su reutilización.

- Implementar decoradores y generadores útiles.

El decorador:

```
# 1. Decorador con *args y **kwargs (Lógica de reintentos)
def reintentar(intentos_max: int = 3, retraso: float = 0.5) -> Callable:
    """Decorador que reintentará ejecutar una función si lanza una excepción"""
    # Simulación de función inestable decorada
    @reintentar(intentos_max=2, retraso=0.2)
    def operacion_riesgosa(datos: dict) -> dict:
```

Agrega una funcionalidad adicional a `operacion_riesgosa`, si la operación falla, intenta ejecutarla nuevamente hasta alcanzar el número máximo de intentos, de esa manera aplicamos el principio de separación de responsabilidades.

`*args` y `**kwargs` aparecen aquí:

```
def wrapper(*args: Any, **kwargs: Any) -> Any:
```

Esto permite que el decorador sea reutilizable con funciones que tengan diferentes parámetros, después en:

```
return func(*args, **kwargs)
```

Se reenvían esos argumentos a la función original, permitiendo que el decorador funcione como una interfaz transparente adaptada a cualquier función, sin importar cuántos parámetros reciba o de qué tipo sean.

El uso de `@wraps` dentro del decorador:

```
@wraps(func)
```

Asegura que la función decorada conserve su nombre original, su docstring y metadatos, evitando que la API de la función sea enmascarada por el wrapper.

El generador en:

```
def procesar_stream_datos(total_registros: int) -> Generator[dict]:
    """Genera datos secuencialmente usando yield sin cargar todo en RAM."""
    for i in range(1, total_registros + 1):
        yield {
            "id": i,
            "sku": f"PROD-{i:03d}",
            "monto": float(i * 10.5),
            "activo": i % 2 == 0,
        }
```

El cual utiliza yield en lugar de return, hace que los registros se produzcan uno por uno, conforme se ocupan, es decir no crea necesariamente todos los registros en memoria al mismo tiempo.

- Crear context managers para recursos.

La implementación de context manager:

```
@contextmanager
def medir_tiempo(etiqueta: str) -> Generator[None]:
```

Se usa aquí:

```
with medir_tiempo("Procesamiento de Stream"):
    for registro in activos:
```

Mide cuánto tarda en ejecutarse el bloque. La librería contextlib transforma un generador en un administrador de contexto sintácticamente limpio, evitando implementar manualmente la clase con dunder methods (___enter___ y ___exit___).

Laboratorio

- Decorador de reintentos con backoff.

El decorador permite volver a ejecutar automáticamente una función cuando ocurre una excepción, utiliza un backoff exponencial, así que el tiempo de espera aumenta después de cada intento fallido.

```
9 def reintentar(intentos_max: int = 3, retraso: float = 0.5) -> Callable:
10     """Decorador que reintenta ejecutar una función si lanza una excepción."""
11
12     def decorador(func: Callable) -> Callable:
13         @wraps(func)
14         def wrapper(*args: Any, **kwargs: Any) -> Any:
15             for intento in range(1, intentos_max + 1):
16                 try:
17                     return func(*args, **kwargs)
18                 except Exception as err:
19                     print(
20                         f"[Intento {intento}/{intentos_max}] Error en {func.__name__}: {err}"
21                     )
22                     if intento == intentos_max:
23                         raise
24
25                     espera = retraso * (
26                         2 ** (intento - 1)
27                     ) # Modificación para backoff.
28                     print(f"Reintentando en {espera:.2f} segundos...")
29                     time.sleep(espera)
```

```
76 @reintentar(intentos_max=2, retraso=0.2)
```

Utiliza un ciclo for para controlar los intentos (línea 15), si la función falla (línea 18) calcula cuánto debe esperar (línea 25-27). La característica de backoff exponencial es que el tiempo va aumentando, primer reintento 0.2 segundos, segundo reintento 0.4 segundos y así sucesivamente, se utiliza para cuando una operación falla por algo temporal, no se

quiere que intente de inmediato una y otra vez. El backoff permite darle tiempo al servicio para recuperarse.

- **Generador por lotes y context manager de temporización.**

Un generador por lotes es el que produce los datos de forma perezosa, pero en lugar de devolver un registro individual, agrupa varios registros y los entrega como un lote.

```
49 def procesar_lotes_datos(  
50     total_registros: int, tamano_lote: int = 2  
51 ) -> Generator[list[dict]]:  
52  
53     lote = []  
54  
55     for i in range(1, total_registros + 1):  
56  
57         lote.append(  
58             {  
59                 "id": i,  
60                 "sku": f"PROD-{i:03d}",  
61                 "monto": float(i * 10.5),  
62                 "activo": i % 2 == 0,  
63             }  
64         )  
65  
66         if len(lote) == tamano_lote:  
67             yield lote  
68             lote = []  
69  
70     # Entregar el último lote aunque esté incompleto  
71     if lote:  
72         yield lote
```

Aquí, en la lista temporal (línea 53) cada registro se agrega (línea 57) cuando alcanza el tamaño establecido (línea 66-68) y al final permite entregar el último lote aunque tenga menos elementos (línea 71 y 72). Con yield hacemos que los datos se produzcan conforme se necesitan, en lugar de crear todos los registros y guardarlos en memoria al mismo tiempo.

El context manager de temporización mide cuánto tiempo tarda en ejecutarse un bloque de código. Lo definimos mediante:

```
37 @contextmanager  
38 def medir_tiempo(etiqueta: str) -> Generator[None]:  
39     """Context manager para medir el tiempo de ejecución de un bloque de código."""  
40     inicio = time.perf_counter()  
41     try:  
42         yield  
43     finally:  
44         fin = time.perf_counter()  
45         print(f" [{etiqueta}] Tiempo transcurrido: {fin - inicio:.4f} segundos")
```

Primero registra el tiempo inicial (línea 40), después yield (línea 42) permite que se ejecute el código que está dentro del with:

```
88     with medir_tiempo("Procesamiento de Stream"):
89         for lote in stream:
90             for registro in lote:
91                 if registro["activo"]:
92                     try:
93                         res = operacion_riesgosa(registro)
94                         print(f"Éxito: {res}")
95                     except ValueError as e:
96                         print(f"Fallo definitivo: {e}")
97
98
99
100
```

Al terminar, calcula el tiempo final (línea 44), lo obtiene y finalmente lo muestra (línea 45); el finally (línea 43) es importante porque garantiza que la medición se realice incluso si ocurre una excepción dentro del bloque.